

701. Insert into a BST

PLATFORM

Platform: LeetCode

DIFFICULTY

MEDIUM

DATE

Solved: July 29, 2026

Problem Summary

Given the root of a BST and a value to insert, insert the value and return the root of the updated tree.
Guaranteed the value doesn't already exist.

Approach

Recurse down the BST like a search — when you hit null, that's the correct insertion point. Return the new node upward; parent attaches it via `root.left = ...` or `root.right = ...`.

Algo

1. If `root == null` return `new TreeNode(val)` — insert here.
2. If `val < root.val` → `root.left = insertIntoBST(root.left, val)`.
3. Else → `root.right = insertIntoBST(root.right, val)`.
4. Return `root`.

Observations

- BST insertion never restructures existing nodes — only adds a new leaf.
- return root at the end is essential — it propagates unchanged nodes back up the call stack.
- This pattern (`root.left = recurse(root.left)`) is reused in deletion and many other tree modification problems.

Time Complexity & Space Complexity

$O(h)$ — traverses one path from root to leaf.

$O(h)$ — recursion stack.

Key Insights

- Insertion always happens at a **leaf position** — you never insert in the middle of the tree.
- `root.left = insertIntoBST(root.left, val)` — the return value is assigned back, propagating the new node up the call stack cleanly.
- Same skeleton as search — only base case changes from return null to return new `TreeNode(val)`.

701. Insert into a Binary Search Tree Solved

Medium Topics Companies

You are given the `root` node of a binary search tree (BST) and a value to insert into the tree. Return the `root` node of the BST after the insertion. It is guaranteed that the new value does not exist in the original BST.

Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return any of them.

Example 1:

Input: `root = [4,2,7,1,3], val = 5`
Output: `[4,2,7,1,3,5]`
Explanation: Another accepted tree is:

Runtime: 0 ms | Beats 100.00%
Memory: 46.89 MB | Beats 86.87%

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  *     TreeNode(int val, TreeNode left, TreeNode right) {
10 *         this.val = val;
11 *         this.left = left;
12 *         this.right = right;
13 *     }
14 * }
15 */
16
17 class Solution {
18     public TreeNode insertIntoBST(TreeNode root, int val) {
19         if (root == null) return new TreeNode(val);
20
21         if (val < root.val) root.left = insertIntoBST(root.left, val);
22         else root.right = insertIntoBST(root.right, val);
23
24         return root;
25     }
26 }
```

Testcase: Test Result
Accepted Runtime: 0 ms
Case 1 Case 2 Case 3
Input: `root = [4,2,7,1,3]`